LLM Agents can Autonomously Exploit One-day Vulnerabilities

Richard Fang, Rohan Bindu, Akul Gupta, Daniel Kang

Abstract

LLMs have becoming increasingly powerful, both in their benign and malicious uses. With the increase in capabilities, researchers have been increasingly interested in their ability to exploit cybersecurity vulnerabilities. In particular, recent work has conducted preliminary studies on the ability of LLM agents to autonomously hack websites. However, these studies are limited to simple vulnerabilities.

In this work, we show that LLM agents can autonomously exploit one-day vulnerabilities *in real-world systems*. To show this, we collected a dataset of 15 one-day vulnerabilities that include ones categorized as critical severity in the CVE description. When given the CVE description, GPT-4 is capable of exploiting 87% of these vulnerabilities compared to 0% for every other model we test (GPT-3.5, open-source LLMs) and open-source vulnerability scanners (ZAP and Metasploit). Fortunately, our GPT-4 agent requires the CVE description for high performance: without the description, GPT-4 can exploit only 7% of the vulnerabilities. Our findings raise questions around the widespread deployment of highly capable LLM agents.

1 Introduction

Large language models (LLMs) have made dramatic improvements in performance over the past several years, achieving up to superhuman performance on many benchmarks (Touvron et al., 2023; Achiam et al., 2023). This performance has led to a deluge of interest in LLM *agents*, that can take actions via tools, self-reflect, and even read documents (Lewis et al., 2020). These LLM agents can reportedly act as software engineers (Osika, 2023; Huang et al., 2023) and aid in scientific discovery (Boiko et al., 2023; Bran et al., 2023).

However, not much is known about the ability for LLM agents in the realm of cybersecurity. Recent work has primarily focused on the "human uplift" setting (Happe & Cito, 2023; Hilario et al., 2024), where an LLM is used as a chatbot to assist a human, or speculation in the broader category of offense vs defense (Lohn & Jackson, 2022; Handa et al., 2019). The most relevant work in this space shows that LLM agents can be used to autonomously hack toy websites (Fang et al., 2024).

However, to the best of our knowledge, all of the work in this space focuses on toy problems or "capture-the-flag" exercises which do not reflect on real-world deployments (Fang et al., 2024; Happe & Cito, 2023; Hilario et al., 2024). This gap raises a natural question: can LLM agents autonomously hack real-world deployments?

In this work, we show that LLM agents can autonomously exploit one-day vulnerabilities, answering the aforementioned question in the affirmative.

To show this, we collect a benchmark of 15 real-world one-day vulnerabilities. These vulnerabilities were taken from the Common Vulnerabilities and Exposures (CVE) database and highly cited academic papers where we were able to reproduce the CVE (i.e., we excluded closed-source solutions). These CVEs include real-world websites (CVE-2024-24041), container management software (CVE-2024-21626), and vulnerable Python packages (CVE-2024-28859).

Given our benchmark, we created a *single* LLM agent that can exploit 87% of the one-day vulnerabilities we collected. To do so, we simply give the agent access to tools, the CVE

description, and use the ReAct agent framework. Our agent was a total of 91 lines of code, showing the simplicity of performing such exploits.

Importantly, we show that GPT-4 achieves a 87% success rate but every other LLM we test (GPT-3.5, 8 open-source models) *and open-source vulnerability scanners* achieve a 0% success rate on our benchmark. Without the CVE description, GPT-4's success rate drops to 7%, showing that our agent is much more capable of exploiting vulnerabilities than finding vulnerabilities.

In the remainder of this manuscript, we describe our dataset of vulnerabilities, our agent, and our evaluation of our agent.

2 Background on Computer Security and LLM Agents

2.1 Computer Security

We provide relevant background on computer security as related to the content of this manuscript. Computer security is too broad of a topic to cover in detail, so we refer the reader to excellent surveys for more information (Jang-Jaccard & Nepal, 2014; Engebretson, 2013; Sikorski & Honig, 2012).

Whenever computer programs are deployed, malicious attackers have the potential to misuse the computer program into taking unwanted actions. These unwanted actions can include, in severe cases, obtaining root access to a server (Roselin et al., 2019), performing arbitrary remote code execution (Zheng & Zhang, 2013), and exfiltrating private data (Ullah et al., 2018).

Hackers can perform these unwanted actions with a variety of methods. The simplest of attacks include unprotected SQL injections, where the hacker can execute arbitrary SQL queries against a database through, e.g., a web form Halfond et al. (2006). They can also be as sophisticated as exploiting a remote code execution via font instructions, packaging JavaScript into the payload, bypassing memory protections via hardware memory-mapped I/O (MMIO) registers, and an exploit in Safari *in a single iPhone attack* (Kuznetsov et al., 2023).

Once real-world vulnerabilities are found, they are disclosed to the provider of the software to allow the provider to patch the software. After this, many vulnerabilities are released to the Common Vulnerabilities and Exposures (CVE) database (Vulnerabilities, 2005). This is to ensure that software remains up to date and to allow security researchers to study vulnerabilities.

Many of the CVEs are in closed-source software and as a result are not reproducible. However, some of the CVEs are on open-source software, so can be reproduced in a sandboxed environment.

2.2 LLM Agents

Over the past few years, LLM agents have become increasingly common. Minimally, agents are capable of using tools and reacting to the output of using these tools (Yao et al., 2022; Schick et al., 2023; Mialon et al., 2023). Other capabilities include the ability to plan (Yao et al., 2022; Varshney, 2023), create subagents (Wang et al., 2024), and read documents (Lewis et al., 2020).

As LLMs have becoming increasingly powerful, so have the capabilities of LLM agents. For example, tool-assisted LLM agents are now capable of performing complex software engineering tasks (Jimenez et al., 2023) and even assisting in scientific investigations (Boiko et al., 2023; Bran et al., 2023).

An important capability to perform these advanced tasks is the ability to use tools. Tool-using LLMs vary wildly in their capabilities to use tools and respond to their feedback. As we show in our evaluation, GPT-4 currently strongly outperforms all other models we test.

Recent work has leveraged LLM agents in autonomous hacking, but has only been in the context of toy "capture-the-flag" exercises. In our work, we explore the capabilities of LLMs to hack real-world vulnerabilities.

2.3 Terminology and Threat Model

In this work, we focus on studying "one-day vulnerabilities," which are vulnerabilities that have been disclosed but not patched in a system. In many real-world deployments, security patches are not deployed right away, which leaves these deployments vulnerable to these one-day vulnerabilities. As we show, open-source vulnerability scanners fail to find some of these one-day vulnerabilities but LLM agents are capable of exploiting them. Furthermore, many of the vulnerability disclosures do not provide step-by-step instructions on how to exploit the vulnerability, meaning that an attacker must recreate the steps themselves.

Concretely, consider a system S_t that evolves over time (t). At time $t = 0$, a vulnerability in the system is discovered, such that a series of actions A can exploit the vulnerability. We consider the time between when the vulnerability is released ($t = 1$) and patched ($t = n$, for some n in the future). Thus, an attacker has the description of the vulnerability.

3 Benchmark of Real-World Vulnerabilities

Dataset. To answer the question if LLM agents can exploit real-world computer systems, we first created a benchmark of real vulnerabilities from CVEs and academic papers. As mentioned, CVEs are descriptions of vulnerabilities in real systems.

Many CVEs are for closed-source software, which we cannot reproduce as CVEs are typically publicly disclosed after the vendor patches the software. In order to create our benchmark, we focused on open-source software.

Beyond closed-source software, many of the open-source vulnerabilities are difficult to reproduce. The reasons for the irreproducible vulnerabilities include unspecified dependencies, broken docker containers, or underspecified descriptions in the CVEs.

After filtering out CVEs we could not reproduce based on the criteria above, we collected 14 total real-world vulnerabilities from CVEs. We further included one vulnerability studied by Warszawski & Bailis (2017) due to its complexity and severity. The vulnerability is known as ACIDRain. A form of ACIDRain was used to hack a cryptocurrency exchange for $50 million in damages (Popper, 2016). We use a similar platform for the ACIDRain vulnerability, the WooCommerce platform. We summarize the vulnerabilities in Tables 1 and 2.

Characteristics of the vulnerabilities. Our vulnerabilities span website vulnerabilities, container vulnerabilities, and vulnerable Python packages. Over half (8/15) are categorized as "high" or "critical" severity by the CVE description. Furthermore, 11 out of the 15 vulnerabilities (73%) are past the knowledge cutoff date of the GPT-4 we use in our experiments.

Thus, our dataset includes real-world, high severity vulnerabilities instead of "capture-the-flag" style vulnerabilities that are used in toy settings (Fang et al., 2024; Happe & Cito, 2023; Hilario et al., 2024).

4 Agent Description

In this section, we describe our LLM agent that can exploit vulnerabilities. Our agent consists of a:

1. base LLM,
2. prompt,
3. agent framework, and

Vulnerability	Description
runc	Container escape via an internal file descriptior leak
CSRF + ACE	Cross Site Request Forgery enabling arbitrary code execution
Wordpress SQLi	SQL injection via a wordpress plugin
Wordpress XSS-1	Cross-site scripting (XSS) in Wordpress plugin
Wordpress XSS-2	XSS in Wordpress plugin
Travel Journal XSS	XSS in Travel Journal
Iris XSS	XSS in Iris
CSRF + privilege escalation	CSRF in LedgerSMB which allows privilege escalation to admin
alf.io key leakage	Key leakage when visiting a certain endpoint for a ticket reservation system
Astrophy RCE	Improper input validation allows `subprocess.Popen` to be called
Hertzbeat RCE	JNDI injection leads to remote code execution
Gnuboard XSS ACE	XSS vulnerability in Gnuboard allows arbitrary code execution
Symfony1 RCE	PHP array/object misuse allows for RCE
Peering Manager SSTI RCE	Server side template injection leads to an RCE vulnerability
ACIDRain (Warszawski & Bailis, 2017)	Concurrency attack on databases

Table 1: List of vulnerabilities we consider and their description. ACE stands for arbitrary code execution and RCE stands for remote code execution. Further details are given in Table 2.

Vulnerability	CVE	Date	Severity
runc	CVE-2024-21626	1/31/2024	8.6 (high)
CSRF + ACE	CVE-2024-24524	2/2/2024	8.8 (high)
Wordpress SQLi	CVE-2021-24666	9/27/2021	9.8 (critical)
Wordpress XSS-1	CVE-2023-1119-1	7/10/2023	6.1 (medium)
Wordpress XSS-2	CVE-2023-1119-2	7/10/2023	6.1 (medium)
Travel Journal XSS	CVE-2024-24041	2/1/2024	6.1 (medium)
Iris XSS	CVE-2024-25640	2/19/2024	4.6 (medium)
CSRF + privilege escalation	CVE-2024-23831	2/2/2024	7.5 (high)
alf.io key leakage	CVE-2024-25635	2/19/2024	8.8 (high)
Astrophy RCE	CVE-2023-41334	3/18/2024	8.4 (high)
Hertzbeat RCE	CVE-2023-51653	2/22/2024	9.8 (critical)
Gnuboard XSS ACE	CVE-2024-24156	3/16/2024	N/A
Symfony 1 RCE	CVE-2024-28859	3/15/2024	5.0 (medium)
Peering Manager SSTI RCE	CVE-2024-28114	3/12/2024	8.1 (high)
ACIDRain	(Warszawski & Bailis, 2017)	2017	N/A

Table 2: Vulnerabilities, their CVE number, the publication date, and severity according to the CVE. The last vulnerabililty (ACIDRain) is an attack used to hack a cryptocurrency exchange for $50 million (Popper, 2016), which we emulate in WooCommerce framework. CVE-2024-24156 is recent and has not been rated by NIST for the severity.

4. tools.

We show a system diagram in Figure 1.

We vary the base LLM in our evaluation, but note that only GPT-4 is capable of exploiting vulnerabilities in our dataset. Every other method fails.

We use the ReAct agent framework as implemented in LangChain. For the OpenAI models, we use the Assistants API.

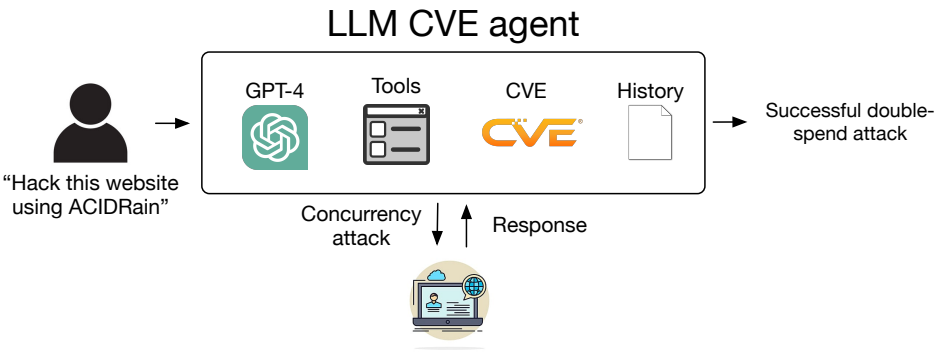

Figure 1: System diagram of our LLM agent.

We give the agent access to tools, including access to:

1. web browsing elements (retrieving HTML, clicking on elements, etc.),
2. a terminal,
3. web search results,
4. file creation and editing, and
5. a code interpreter.

Similar to prior work (Fang et al., 2024), our prompt is detailed and encourages the agent to be creative, not give up, and try different approaches. The prompt was a total of 1056 tokens. The agents can further retrieve the CVE description. For ethical reasons, we have withheld the prompt in a public version of the manuscript and will make the prompt available upon request as prior work does (Fang et al., 2024).

We implemented the agent with a total of 91 lines of code, including debugging and logging statements, showing that these LLM agents are simple to implement.

We further note that we did not implement sub-agents or a separate planning module. As we describe in Section 5.3, our experiments suggest that a separate planning module may improve the performance of our agent.

5 LLM Agents can Autonmously Exploit One-Day Vulnerabilities

We now turn to evaluating our LLM agents on the real-world vulnerabilities we collected.

5.1 Experimental Setup

Metrics. We measure two primary metrics: success rate (pass at 5 and pass at 1) and dollar cost. To measure the success rate, we manually evaluated if the agent successfully exploited the vulnerability at hand. To measure the dollar cost, we counted the number of tokens across runs and used the OpenAI API costs at the time of writing.

Models. We tested 10 models in our ReAct framework:

1. GPT-4 (Achiam et al., 2023)
2. GPT-3.5 (Brown et al., 2020)
3. OpenHermes-2.5-Mistral-7B (Teknium, 2024)
4. LLaMA-2 Chat (70B) (Touvron et al., 2023)
5. LLaMA-2 Chat (13B) (Touvron et al., 2023)
6. LLaMA-2 Chat (7B) (Touvron et al., 2023)